A Data-driven approach for investigating Airbnb listing analysis in Nashville

Wasim Raja Mondal

March 03, 2026

Nashville is a major business, economic, and political hub in Tennessee. Airbnb plays an important role in Nashville's economic growth. In this work, we are analyzing Nashville's Airbnb data. Based on our data analysis approach, we aim to answer important questions: (1) What are the key factors controlling Airbnb listing prices? Can we build a model to predict listing prices? (2) Can we develop another model that helps to predict whether room booking is available or not?

We have organized our data-driven approach as follows. We discuss the business understanding of our analysis in Section 1. In Section 2, we describe how we will understand the data and gain some insight from its initial visualization. We outline a brief plan for preparing the dataset for predictive modeling in Section 3. In Section 4, we outline the testing types, techniques, and models we have developed. Here, we also discuss how we validate the model by calculating appropriate metrics. Finally, we discuss the application of the developed and validated model to the unknown dataset. An outline of our plan is shown in Figure 1.

1 Business Understanding

In this work, we aim to identify the key factors that determine Airbnb listing prices. For example, we would like to understand how room type, characteristics, and amenities affect listing prices. Also, we will highlight the factors that impact review ratings and guest satisfaction, and how availability and booking convenience relate to demand patterns. We believe that all this knowledge will be beneficial to both the guest and the host from a business perspective.

2 Data Understanding

The backbone of every data-driven approach is the quality of data we have. The Nashville Airbnb dataset offers very high-quality data. In particular, the dataset includes 7 files containing structured tabular data on hosts, listings, calendars, reviews, and more. The relationships among the various tabular data and the corresponding ERD diagram for our dataset are shown in Figure 1.

Figure 1: Schematic of the Data-driven approach for investigating crime in the Canton neighborhood of Michigan

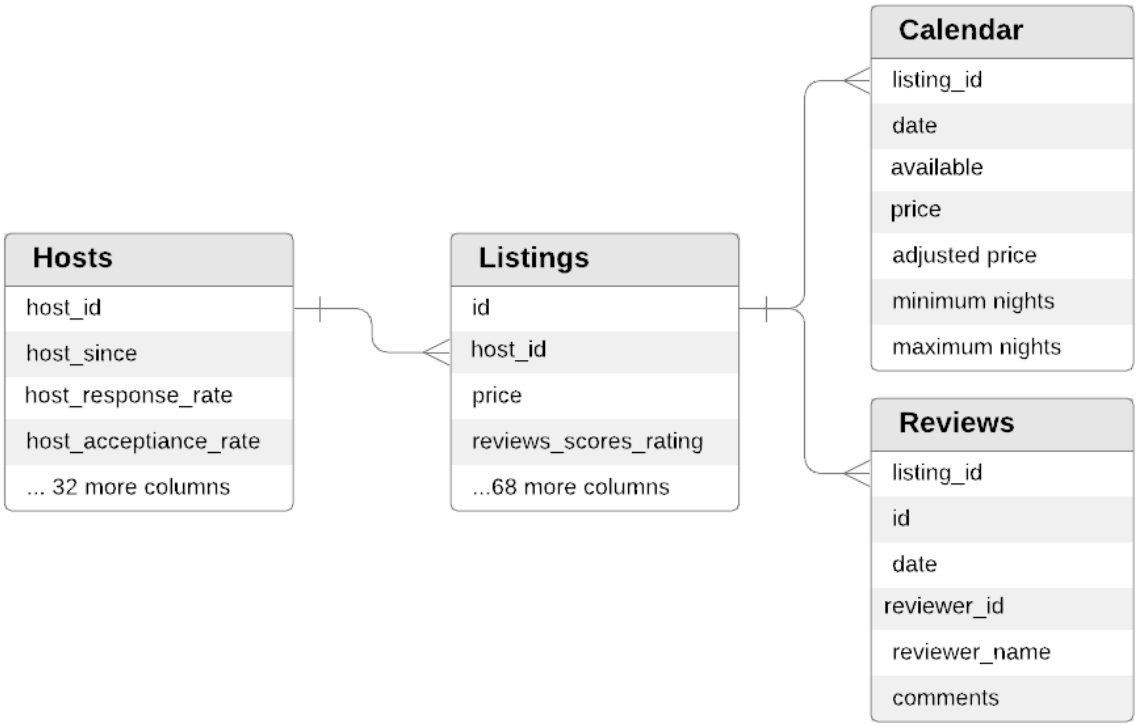

Figure 2: Nashville Airbnb ERD diagram

To gain insight from initial observations, we present various plots from the dataset. For example, in Figure 3, we show the top 10 amenities across the dataset, ranked by their importance in controlling listing prices. In particular, we find that Airbnb rooms with a crib, a Lockable Bedroom, and an EV charger have significantly higher prices. However. Wifi, Carbon monoxide alarm, first aid kit, TV, highchair, pool, gym, also significant features control prices. As Figure 4 suggests, the average price of a room without a pool is around $200, whereas the price of a room with a pool is, on average, more than $300.

In addition, we look into which kinds of rooms are most often listed. From Figure 5, we see that most of the time, the entire home/apt is listed, whereas shared listings are not very popular, as they are listed very infrequently. We have also extracted some information regarding room type. For example, there are a total of 1646678 Entire homes/apt, 196024 private rooms, 22267 hotel rooms, and 4747 shared rooms in this Airbnb Nashville dataset.

As expected, the cleanliness score directly affects on overall rating. From Figure 6, it is very clear that the higher the cleanliness score, the higher the overall rating. For example, the cleanliness score of 2 has a rating of 60, whereas the cleanliness score of 10 has an overall rating of more than 100.

Instant bookable or not, it also significantly impacts price. As shown in Figure 7, the average price of non-instant bookable rooms is close to $200, whereas the average price of instant bookable rooms is more than $ 200.

After gaining some basic insights from the dataset, we move on to the next section: data preparation for our model.

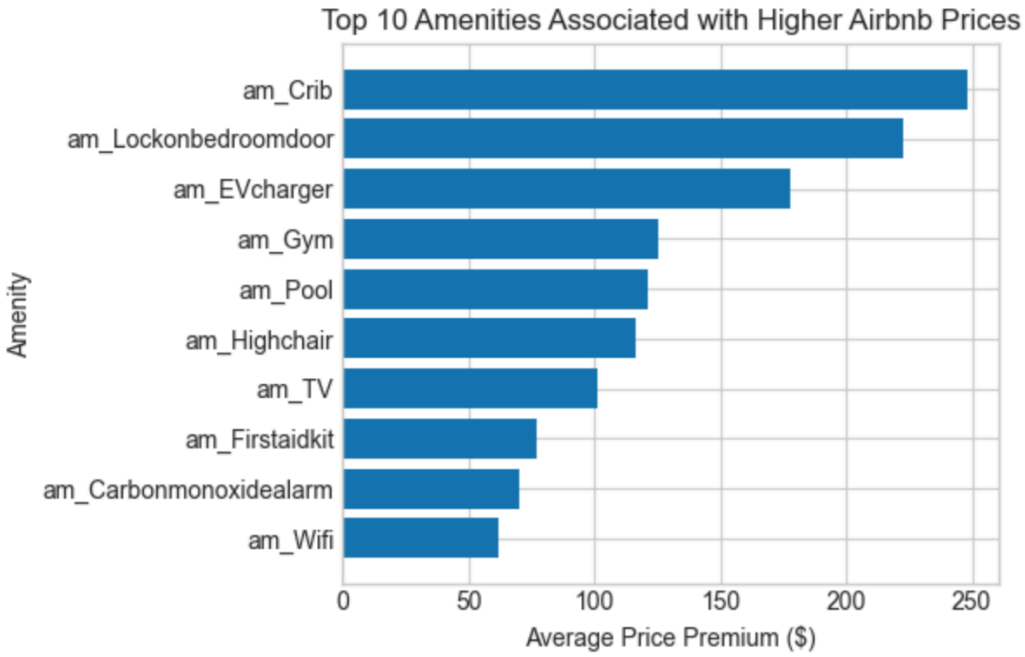

Figure 3: Top 10 Amenities associated with Higher Airbnb prices.

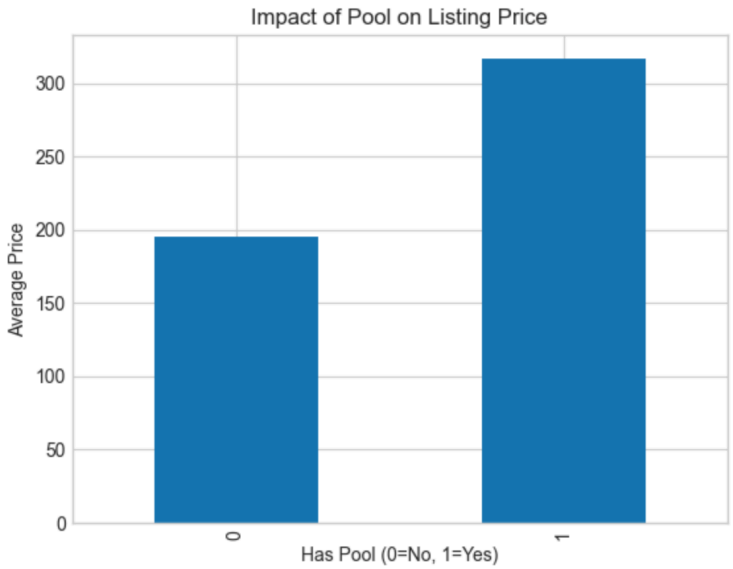

Figure 4: Impact of pool on listing price.

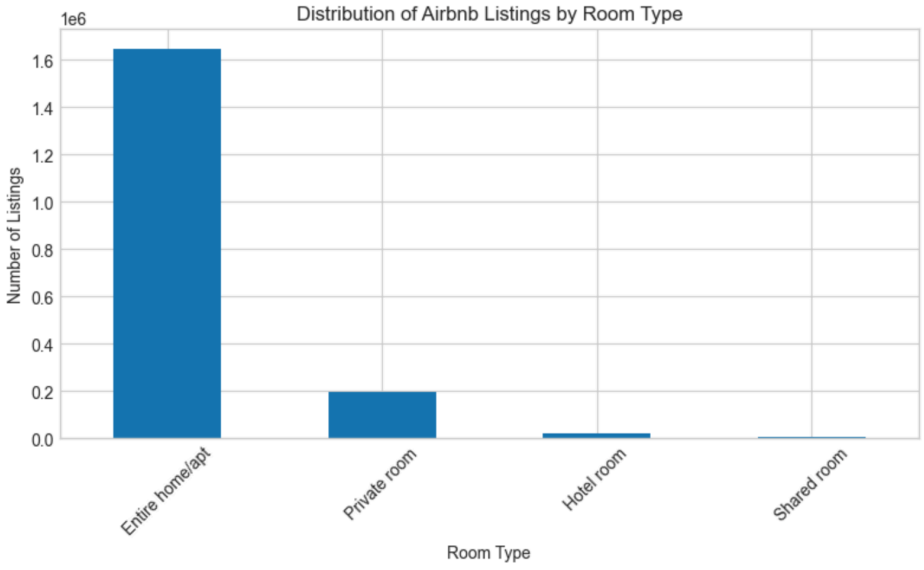

Figure 5: Distribution of Airbnb listings by room type.

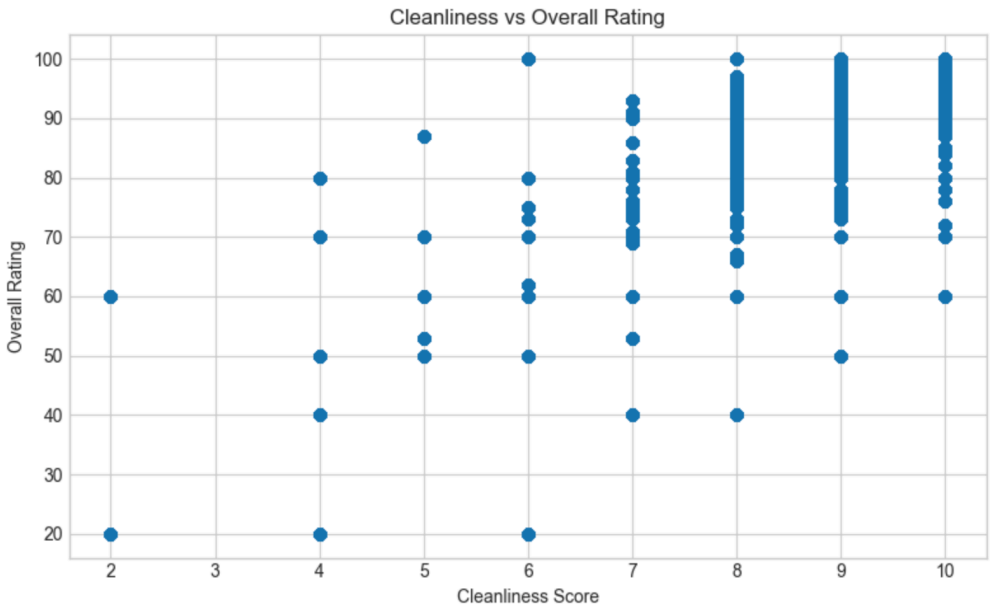

Figure 6: Effect of Cleanliness on Overall Rating

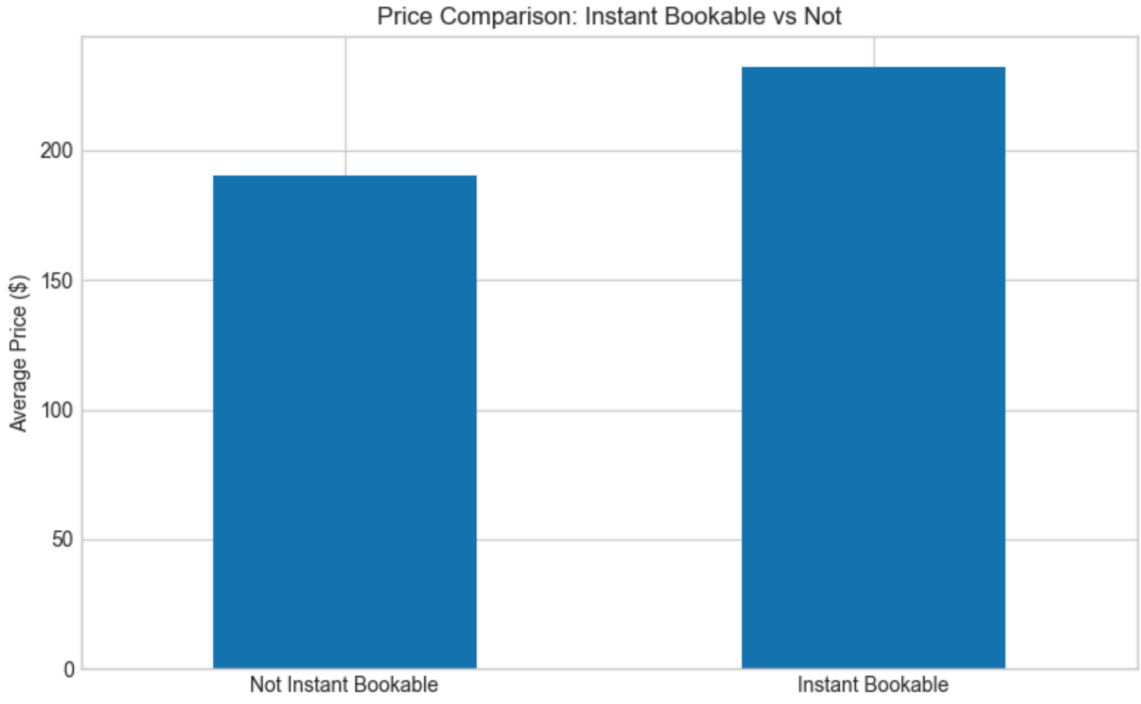

Figure 7: Comparison of price with instant and non-instant bookng

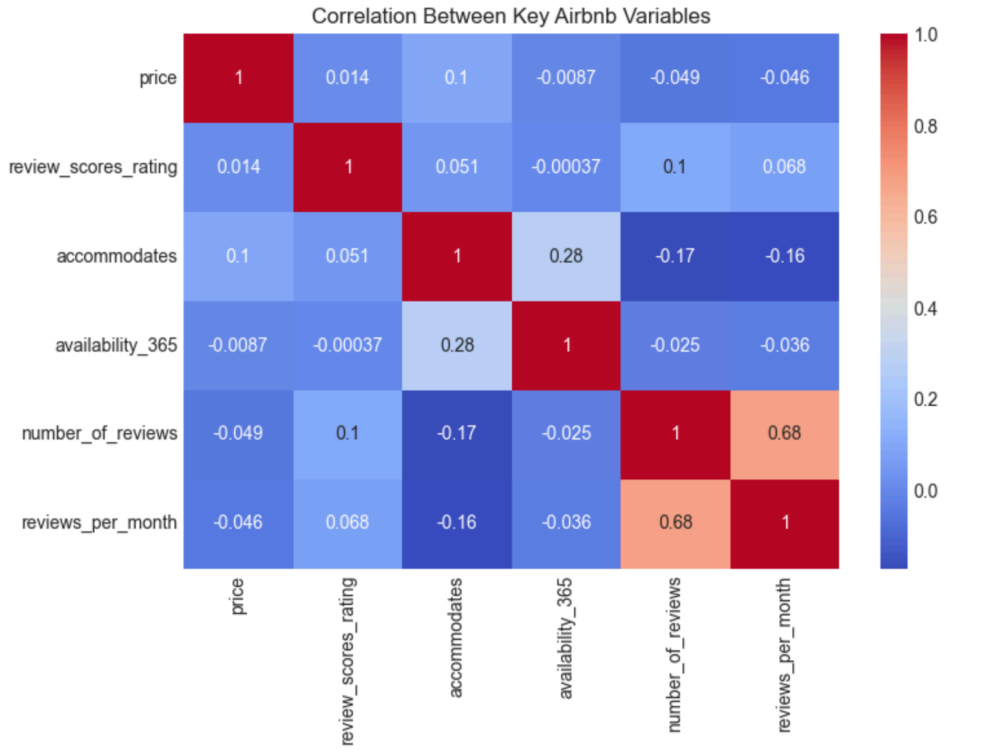

Figure 8: Correlation between key Airbnb variables

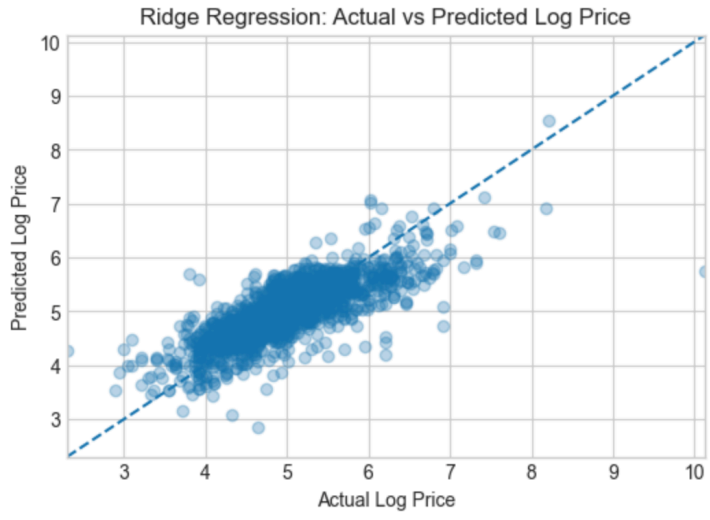

Figure 8: Scattering plot between the actual value and predicted value of price in logscale obtained from Ridge regression.

	Actual_Price	Predicted_Price	Absolute_Error	Percent_Error
0	174.0	224.530713	50.530713	29.040639
1	299.0	148.530503	150.469497	50.324246
2	60.0	99.656064	39.656064	66.093440
3	282.0	286.733322	4.733322	1.678483
4	240.0	245.715595	5.715595	2.381498
5	107.0	88.118440	18.881560	17.646318
6	564.0	418.893685	145.106315	25.728070
7	178.0	236.681291	58.681291	32.967017
8	265.0	298.168644	33.168644	12.516469
9	386.0	688.056384	302.056384	78.252949

Table 1: A comparison between the actual and predicted values of price with error values.

	Actual_Price	Predicted_Price	Absolute_Error	Percent_Error
1515	10.0	71.353939	61.353939	613.539392
459	45.0	295.295975	250.295975	556.213277
292	50.0	268.563368	218.563368	437.126736
1623	22.0	87.117593	65.117593	295.989059
771	20.0	74.176195	54.176195	270.880975
1026	45.0	133.358575	88.358575	196.352388
1483	100.0	296.018992	196.018992	196.018992
1390	99.0	285.448230	186.448230	188.331546
1310	411.0	1177.414217	766.414217	186.475478
1379	47.0	134.061759	87.061759	185.237785

Table 2: A comparison between the actual and predicted values of price, with the error values with the highest error information on top order.

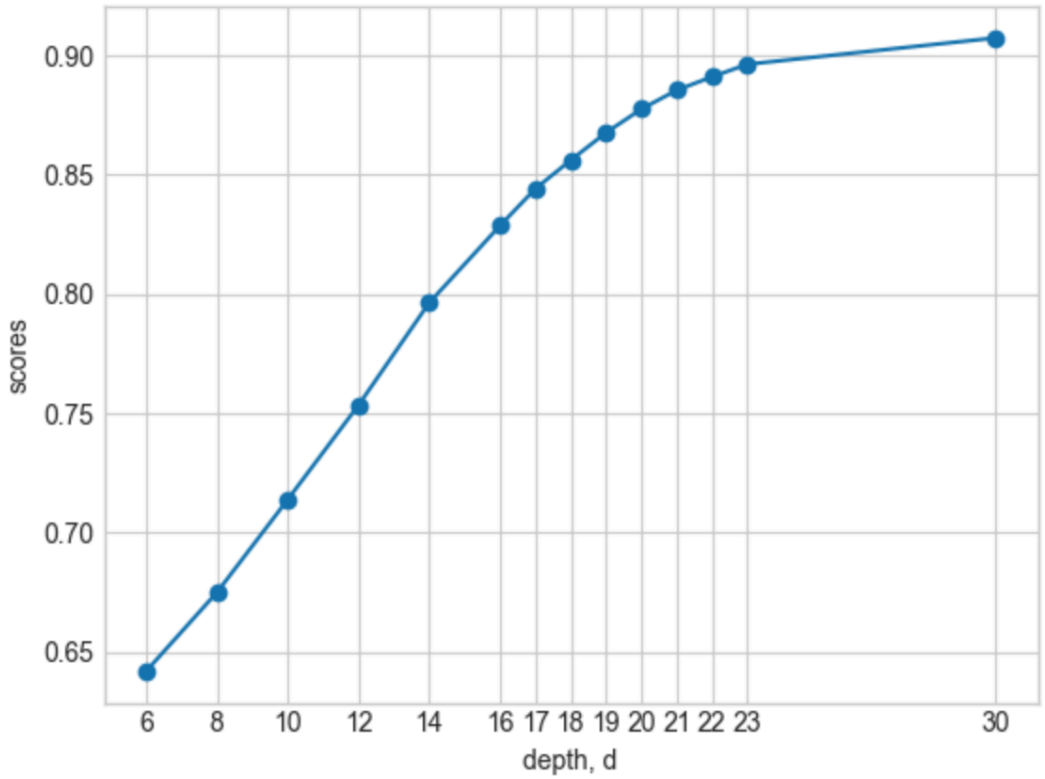

Figure 9: Variation of the Model's score with changing in depth.

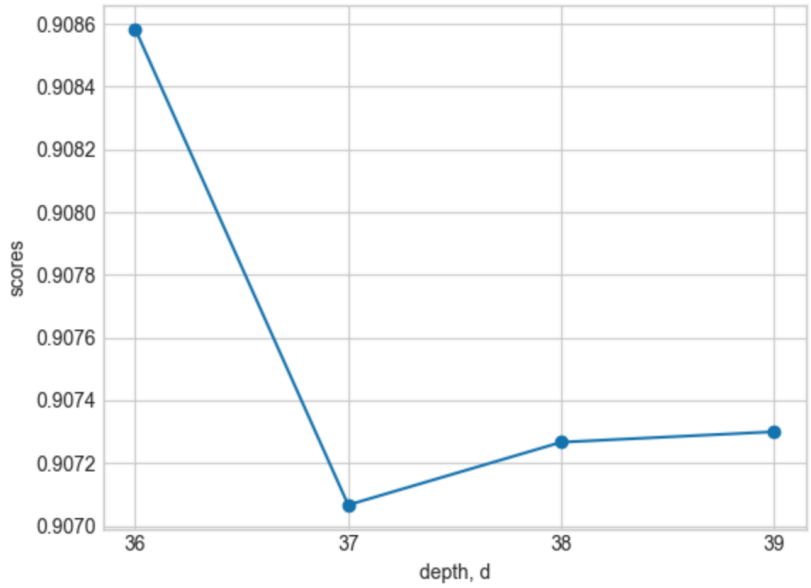

Figure 10: Variation of the Model's score with changes in depth.

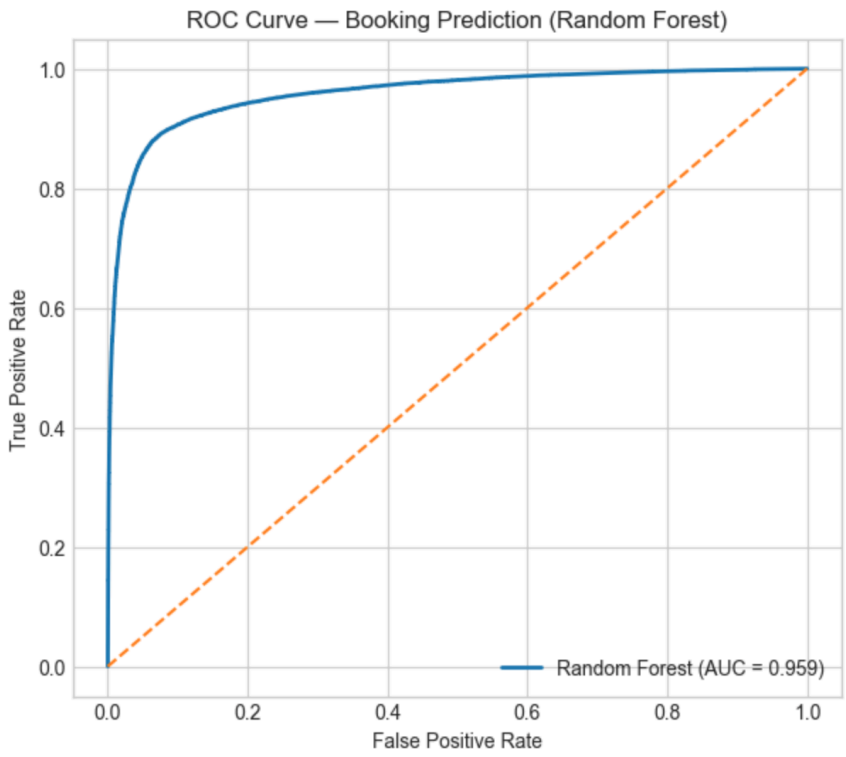

Figure 11: ROC Curve for the Booking prediction using Random Forest Classifier

Actual_is_booked	Predicted_is_booked	Predicted_Probability
0	0	0.442637
0	0	0.135000
1	1	0.865000
0	0	0.307500
1	1	0.919531
1	1	0.809157
0	1	0.545000
1	1	0.940000
1	1	0.883722
1	1	0.995000
1	1	0.935000
0	0	0.065000
0	0	0.026608
1	1	0.782026
0	0	0.203333
0	0	0.296875
0	0	0.041667
1	1	0.747917
1	1	0.924000
0	0	0.255894

Table 3: Actual vs predicted comparison obtained from the Random Forest classifier

3 Data Preparation

As we mentioned, there are 7 files in this dataset. They are amenitiesdummies. Csv, cal_listing.csv.gz, calender.csv.gz, hosts.csv, listing.csv.gz, listing_review.csv.gz, reviews.csv.gz.

For our first model, which is a regression model for predicting prices, we construct the dataframe from the data file cal_listing.csv.gz. As we can see from the Jupyter notebook, this dataframe has 222 features in its initial stage, including integer, float, and fraction data types, as well as object data types. Among these 221 features, we construct a new dataframe by extracting only the important features related to location, property characteristics, pricing policy, demand, host info, and the target feature. There are also NAN or missing values in these feature sets. We clean these features by removing them. The important features related to location are “latitude”, ‘longitude’ and ‘neighbourhood_cleansed’. The property characteristics feature is ‘property_type’, ‘room_type’, ‘accommodates’, ‘bedrooms’, ‘beds’, and ‘bathroom_text.’ Pricing-related feature is ‘minimum_nights.’ Our considered demand-related features are ‘avilability_365’, ‘number_of_reviews’, and ‘reviews_per_month’. The host-related feature is ‘host_is_superhost’, ‘instant_bookable’, and the target feature is ‘price’. Among these features, many are object data types that are not model-ready. Such features are ‘neighbourhood_cleansed’, ‘property_type’, and ‘room-type’. We preprocess these categorical features using Scikit-learn's OneHotEncoder. We also scale the data using Scikit-learn's StandScaler.

To investigate the possibility of multicollinearity, we also displayed a correlation-matrix heat map in Figure 8, considering a few important features. As shown in the heat map, the correlations are low. However, there is a significant correlation between feature ‘reviews_per_month’ and ‘number_of_reviews.’ We will test the model's performance during validation by removing one of these two features.

One important thing to note is that the target variable is highly skewed, which is realistic for real-world data. We converted them to a log scale for model development.

We split the dataset into ⅔-⅓, with ⅔ for training and ⅓ for testing.

As a second model, we treat cal_available as the target variable in a classification problem. We use the same data file to construct the dataframe for this problem. As important features, we consider price, property characteristics, features like ‘accommodates’, ‘bedrooms’, ‘beds’, ‘bathrooms’, room type dummies like ‘entire_room/apt’, ‘private_room’, ‘shared_room’, ‘Hotel_room’, review scores features like ‘review_score_rating’, ‘review_score_location’, host quality related features like ‘host_is_superhost’, ‘instant_bookable’, time features like ‘Monday’..’sunday’, luxury

amenities related features ‘pool’, ‘Hottub’, ‘Gym’ and ‘FreeParkingpremises’.

So, from Cal_available a target, we construct a new variable ‘is_booked’. We assign ‘is_booked. When ‘is_booked’=1, the user would see the room is unavailable or already booked. On the other hand, when ‘is_booked = 0, the user would see the room as available.

We also cleaned the dataset by removing missing values.

4 Modeling

After preprocessing the data, we start testing our model. For the regression problem, first, we employ Ridge regression. To optimize model performance, we also utilize a 5-fold grid search. With the optimal alpha value, we find the best R2 scores for the training and test sets are 0.6281 and 0.608, respectively, which suggests that 63% of the variation in the price training data can be explained by our model, and 61% of the variation in the price test data can be explained by our Ridge regression model. The current R2 score comparison between the train and test data also suggests there is no overfitting, because in an overfitting case, the model performs well on the train data but not on the test data. Finally, after the prediction, we convert the price back from the log scale to the original scale using an exponential.

The comparison between the actual (X) and predicted (Y) price values on a log scale obtained from Ridge regression is shown in Fig. 8. In the ideal case, X and Y axis values will collapse with each other on the straight line. The current plot suggests a significant deviation from a straight line, as expected given the moderate R2 value.

To search for a better model, we use the Lasso Regressor, known for its interpretability. We obtain an R2 score almost identical to that from Ridge regression with Lasso. Lasso selected 74 of 93 features and removed 19. Some of the Lasso’s most important selected features are location (‘longitude’), number of bedrooms (‘num_bedrooms’), number of minimum nights (‘num_minimum_nights’), number of bathrooms, neighbourhood cleaned district, and so on.

For a classification problem, whether the user will see a room available or not, first, we apply a logistic regression model, and we find model accuracy is 0.62, with an F1-score for class 0 as 0.66 and for class 1 as 0.58.

To search for a better decision model, we next apply a Random Forest Classifier, and we find a more improved model with the model’s accuracy increasing from 0.62 to 0.81, and

the f1-score also increases from 0.66 (class=0) and 0.58 (class=1) to 0.83 (class=0) and 0.80 (class=1).

After achieving better performance with the Random Forest classifier than with logistic regression, we begin fine-tuning the current Random Forest classifier. First, we search for the optimal depth that maximizes the model's accuracy. As shown in Figures 9 and 10, the score changes significantly with depth, ranging from 6 to 23. After careful observation, we find that depth d=36 yields the highest model score.

After achieving optimal depth, we tune the class weight and find that class weight=None is best, as expected, since the data is not imbalanced. The full classification report with various class weights has been displayed in the Jupyter notebook.

We also tried tuning the maximum number of features. But we did not find an increase in the model's performance.

Our final Random Forest classifier achieves an overall accuracy of approximately 90.8%, suggesting that it efficiently classifies the booking status for the majority of listings in the test dataset. The confusion matrix confirms that **29,518 available listings** and **24,975 booked listings** were correctly identified, while **2,294 available listings** and **3,213 booked listings** were misclassified. The classification report further guarantees balanced performance across both classes, with precision values of **0.90 for available listings (class 0)** and **0.92 for booked listings (class 1)**, and recall values of **0.93 and 0.89**, respectively. The corresponding **F1-scores of 0.91 and 0.90** imply an overall well-balanced trade-off between precision and recall. In addition, the model achieves an excellent **ROC–AUC score of approximately 0.959 (as shown in Figure 11)**, establishing a very strong ability to distinguish between booked and available listings across different probability thresholds. To conclude, our results establish that the Random Forest model provides a reliable and robust framework for predicting booking availability.

5 Deployment

After validating and establishing the model, we apply the model. A comparison of the actual price, predicted price, Absolute error, and percentage error is shown in Table 1. As Table 1 shows, the predicted value at index 3 is in good agreement with the actual value, with a deviation of just $4.73. However, as indexed 9 suggests, there is a strong deviation between the actual and predicted price, with an error of $302. The largest error between model-predicted and actual values is displayed in Table 2.

We also deploy the final Random Forest Model to predict bookings, and the comparison is shown in Table 3. AS we from, the table model performs well to predict both classes.

Conclusions

In this work, we have developed two models based on the Nashville Airbnb data. The first regression model predicts prices, mostly based on amenities. This model is particularly important for the host because it should know whether it has these amenities, and the host can demand the proper price. In the second classification model, we predict whether a room is booked. This model is expected to be very useful for guests because it will help them decide whether advanced booking is needed. Overall, our data-driven analysis helps both guests and hosts make Airbnb more efficient from a business perspective.